



Rapport projet Sokoban

Realisé par: Kevin Issa et Mickael kovel

matricule Kevin: 000514550

matricule Mickael: 000396950

email: kevin.issa@ulb.be et mickael.kovel@ulb.be

janvier 2022

Table des matières

1	Introduction	3
2	Règles du jeu	3
3	les classes	3
3.1	Sokoban.hpp	3
3.2	Case.hpp	4
3.3	Controller.hpp	4
3.4	Displayer.cpp	4
3.5	Game window.hpp	4
4	Les taches réalisées	4
4.1	les taches de bases	4
4.2	Les taches additionnelles	5
4.2.1	Boite de couleurs	5
4.2.2	Boites légères	5
4.2.3	Informations sur les pas	5
4.2.4	Détection d'échecs	5
4.2.5	Teleporteur	5
4.3	Niveau et selection de niveau	6
5	Simulation de l'exécution du code	6
6	Modèle MVC	7
7	Conclusion	7
8	Annexe	7

1. Introduction

La programmation est une ressource nécessitant énormément de temps et de pratique afin d'être maîtrisée, c'est dans ce cadre que les tps et surtout les projets interviennent.

Dès lors, afin de parfaire notre maîtrise du C++ et de s'assurer de la compréhension des tps, un projet consistant à créer un jeu vidéo nous a été donnée.

Ainsi, ce rapport nous permettra de vous expliquer dans un premier temps le but du jeu avec ses contraintes, mais plus important de vous expliquer la manière dont nous avons procédé pour mener à bien ce projet.

Ensuite, nous vous expliquerons les fonctionnalités que nous avons pu implémenter, le rôle des classes, leur fonctionnement et leur lien entre elle et le modèle MVC. Pour finir, nous expliquerons le fonctionnement du jeu entre les différentes instances du code et les relations entre elles lorsqu'un utilisateur sélectionne un niveau puis effectue un coup après quelques secondes d'attentes.

2. Règles du jeu

Le jeu en question est Sokoban, un petit jeu où nous contrôlons un personnage qui peut bouger dans 4 directions et dont le but est de mettre des boîtes sur des objectifs en faisant attention de ne pas les bloquer étant donné que l'on peut juste les pousser. Ce jeu devait être réalisé avec la librairie graphique FLTK, qui est celle que nous utilisons en tp, afin d'avoir un peu plus agréable à jouer en plus de mettre en pratique ce que nous avons appris durant l'année.

3. les classes

Avant de passer à l'explication des tâches réalisées, il nous semble nécessaire d'expliquer au préalable l'interface et les relations entre les différentes classes du jeu en fin de projet afin de ne pas devoir le réexpliquer pour chaque nouvelle étape du projet.

3.1. Sokoban.hpp

Il s'agit de la classe principale du jeu, c'est celle qui prend le rôle du modèle dans le modèle MVC. Dès lors, c'est elle qui contient toutes les données et états du jeu et les différents calculs à faire pour que le jeu fonctionne.

Pour cela, elle dispose de méthodes permettant de lire le fichier txt correspondant au niveau et grâce à celui-ci de remplir le plateau de tous les éléments le constituant (caisses, objectifs, murs, etc.) avec les méthodes appropriées pour créer chaque élément ainsi que les listes contenant ces éléments. Chaque liste dispose d'un getter afin que les autres classes puissent consulter ces données.

De plus, la classe contient tout les flags et les méthodes permettant faire respecter les règles du jeu, que ce soit la possibilité pour une caisse ou un joueur de bouger, savoir

si une caisse est bloquée, si la partie est perdue ou si le nombre de pas autorisé a été dépassé. Enfin, c'est cette classe qui permet de passer au prochain niveau ou de le recommencer.

3.2. Case.hpp

Cette classe permet d'instancier, un nouvel objet correspond à une cellule. Pour cela, le constructeur a besoin de la valeur que prend l'objet sur le plateau, sa position, son image, son nom. Par la suite, un getter par chaque paramètre du constructeur est fourni. Cette classe est appelée pour chaque création d'un nouvel objet par Sokoban.

3.3. Controller.hpp

Il s'agit de la partie contrôleur du jeu. En effet, c'est elle qui permet de gérer les inputs du joueur et de vérifier grâce au modèle si le coup demandé est correct selon les règles du jeu pour que par la suite, il modifie la position du joueur dans le modèle. C'est également lui qui, selon l'état enregistré par le modèle, détermine si l'on est dans une situation de défaite ou non et si l'on doit passer au prochain niveau.

3.4. Displayer.cpp

L'une des classes principales pour la vue, en effet, c'est elle qui a pour rôle d'afficher les éléments sur le plateau grâce aux listes des différents éléments fournies par le modèle. C'est également elle qui affiche le menu lorsque le jeu démarre.

3.5. Game window.hpp

L'affichage de la fenêtre du jeu ou du menu est géré par cette classe. C'est grâce à Game_window que la fenêtre s'affiche et que les inputs du player sont gérés. En effet, elle affiche 60 fois par seconde les éléments à afficher tels que le displayer et la fenêtre en plus d'être en attente d'un input du joueur. Lorsqu'elle le détecte, elle l'envoie au contrôleur qui se charge de gérer la requête puis de renvoyer le résultat à la vue qui affiche l'avancement du jeu.

4. Les tâches réalisées

4.1. les tâches de bases

Afin de réaliser en premier lieu le jeu de base avec les 5 tâches de bases, nous avons procédé comme ceci. Le niveau est chargé par load game, qui crée un objet pour chaque élément et les ajoutent dans leur liste respective.

Ensuite, le contrôleur vérifie les mouvements du joueur grâce à la fonction check_move qui, si elle valide le mouvement, l'effectue avec play_move. À chaque mouvement, effectuer le contrôleur vérifie si la condition de victoire a été validée. Si c'est le cas, le contrôleur passe au prochain niveau et l'on revient au début du processus. Si le joueur appuie sur la touche "r" ou au bouton correspondant à n'importe quel moment de la partie, le niveau est réinitialiser.

4.2. Les tâches additionnelles

remarque: l'écran d'accueil a été fait mais ne contient pas de section pour elle car il n'y a pas grand chose à dire.

4.2.1. Boîte de couleurs

Cette tâche fut simple à réaliser étant donné que la manière dont nous regardons si une boîte est bien sur son objectif consiste en parcourant la liste des objectifs le concernant et vérifier que le signe attribué à sa boîte remplace la position initiale de l'objectif. Ce procédé fut appliqué aux boîtes de couleurs en prenant d'autres signes pour les boîtes et leurs objectifs.

4.2.2. Boîtes légères

Ce type de boîte nous a demandé de revoir notre manière dont le `check_move` fonctionne. En effet, de base, la fonction regarde juste la case à côté du joueur et vérifie qu'il s'agit bien d'une case où le joueur peut se déplacer que ce soit normalement ou en poussant une caisse. Cependant, ce type de boîte peut en pousser d'autre. Dès lors, nous devons regarder si la caisse légère n'avait pas une autre caisse légère à côté d'elle et ainsi de suite jusqu'à avoir une case libre grâce à une fonction récursive dérécursiée .

4.2.3. Informations sur les pas

Les informations sur les pas que ce soit le meilleur score, le nombre de pas limité ou le nombre de pas actuel fut implémenter de manière très concise. Effectivement, les 2 premiers types de pas sont instanciés dans le fichier du niveau lors de son chargement et le meilleur score est mis à jour lors de la fin du niveau si celui-ci est amélioré et que le joueur a remporté le niveau. Le nombre courant de pas est mis à jour à chaque mouvement autorisé.

4.2.4. Détection d'échecs

Notre implémentation, pour déterminer si toutes les caisses sont bloquées fut la suivante. Nous parcourons la liste contenant toutes les boîtes et nous demandons à chacune de bouger dans les 4 directions, si au moins deux directions ne peuvent pas être effectuées et qu'elles forment un angle droit, nous considérons la boîte comme étant bloquée. Si le nombre de boîtes bloquées est égal au nombre de boîte totale, le contrôleur considère que le niveau est perdu.

4.2.5. Téléporteur

Afin d'implémenter les téléporteurs, nous avons implémentés une fonction `can_tp` qui vérifie si le joueur est sur un téléporteur et si c'est le cas, il vérifie que l'autre téléporteur n'ait rien dessus. Par la suite, un swap est effectué entre les 2 téléporteurs pour bouger le joueur. Si une caisse est sur l'un des deux, celle-ci ne sera pas swap mais bloquera celui-ci.

4.3. Niveau et selection de niveau

Le controleur, lorsque le joueur finit le niveau ou demande à changer de niveau, demande au modèle de changer le niveau en précisant un certain décalage. Dès lors, si le joueur termine celui-ci, le prochain est chargé, grâce à la fonction `load_game` qui est rappelée, en prenant soin au préalable de veiller à ce que le meilleur score soit mis à jour dans le fichier du niveau, si celui-ci a été battu.

Cependant, si le joueur veut changer le niveau avant de le finir, rien n'est enregistré. Afin de changer de niveau, ce dernier, a la possibilité de cliquer sur les boutons, situés dans le menu en bordure de jeu, correspondant au niveau suivant et précédent.

Les niveaux quant à eux sont stockés dans un fichier dont les informations sont analysées grâce à plusieurs méthodes du modèle, qu'importe leur taille. Celles-ci permettent d'une part, d'initialiser les données du niveau et d'autre part de réécrire dans ce fichier si il faut changer des données sur le fichier.

5. Simulation de l'exécution du code

Lors du chargement du jeu, la classe `Game window` affichera pendant 2 secondes l'écran d'accueil puis passera le flag du menu à `false` pour indiquer qu'il est temps d'afficher le jeu. Dès lors les données du premier niveau sont chargées grâce à `read_level_file` et tous les vecteurs nécessaire sont remplis grâce à `load_game` qui est appelée par la fonction `init`.

Ensuite, le `displayer` affiche 60 fois par seconde tout les éléments constituant le plateau en parcourant leur vecteur respectif et le jeu se met en attente du coup du joueur. Lors du coup du joueur, le contrôleur transmet le coup au modèle, grâce à la méthode `check_move`, qui renverra un booléen selon la validité du coup et qui selon la destination du joueur prendra différentes décisions. Si le joueur va sur un mur ou une caisse bloquée, le coup sera refusé, s'il veut pousser une caisse légère ou une caisse normal et que la case suivante est valide, Sokoban acceptera le coup et passera la main à `play move` qui effectuera le coup sur le plateau. Ce moment est géré entièrement par le contrôleur qui ne fait qu'appeler les méthodes du modèle.

Lorsque le coup est joué, plusieurs fonctions examineront l'état de la partie pour s'assurer que les caisses ne soient pas bloquées ou que le joueur ne se trouve pas sur un téléporteur (si c'est le cas, le jeu prendra la décision de téléporter le joueur ou non en fonction de la disponibilité du téléporteur). Après, cela le jeu se remettra en attente d'un coup du joueur.

Cependant, un peu, avant de se remettre en attente, le contrôleur examine l'état de la partie pour savoir si le niveau est gagné ou perdu, auquel cas il demandera au modèle de charger le second niveau grâce à `next level` ou notifiera visuellement le joueur d'un cas d'échec.

6. Modèle MVC

L'utilisation de ce modèle dans notre code est représenté de cette manière. Le modèle est représenté par Sokoban, contrôleur par controller et la vue par display et game window. Lorsque la vue a besoin d'effectuer un coup, il le demande au contrôleur qui vérifie grâce au modèle et à ses données si cela est possible. Si c'est le cas, il modifiera grâce au modèle la position du joueur qui sera ensuite actualisé graphiquement par la vue.

7. Conclusion

En conclusion, ce projet nous aura permis de bien comprendre les aspects du cours qui peuvent sembler abstrait durant le cours théorique. De plus, celui-ci nous a permis de travailler avec un design pattern pour la première fois en plus de nous entraîner d'ors et déjà pour l'examen.

8. Annexe

L'idée original de l'algorithme du projet est repris de javidx9¹, même si, celui-ci a été profondément modifié.

Aucun code n'a été copié, les seuls lignes communes sont la section :

```
switch(direction): NORTH,SOUTH,EAST,WEST
```

L'assistant nous a confirmé que ce ne sera pas considéré comme du plagiat si tout cela est précise dans le rapport.

¹<https://www.youtube.com/watch?v=l7YEaa2otVE>